

Recursive Types and General Recursion

Stefan Muller

CSE 5095: Types and Programming Languages, Fall 2026
Lecture 12

1 General Recursion

As we saw last week, STLC programs all terminate, which makes it not a very useful language. The reason, as we saw, is that we can't write terms involving self-application, like $(\lambda x. x x) (\lambda x. x x)$ and fixed point combinators like Y . But we can explicitly add a fixed point combinator to the language. When we do this, we generally no longer think of it as STLC, but it becomes more similar to another language, PCF, first considered by Gordon Plotkin in 1977. Historically, PCF has only constants (we'll consider just unit) and functions, but we can add sums and products too to get an extension of STLC.

$$\begin{array}{ll} \text{Types} & \tau ::= \text{unit} \mid \tau \rightarrow \tau \\ \text{Expressions} & e ::= x \mid () \mid \lambda x. \tau e \mid e e \mid \text{fix } x = e \end{array}$$

We just need static and dynamic semantics for fix ; all the other rules are the same as what we've seen before.

$$\frac{\Gamma, x : \tau \vdash e : \tau}{\Gamma \vdash \text{fix } x = e : \tau} \text{ (FIX)} \qquad \frac{}{\text{fix } x = e \mapsto [\text{fix } x = e/x]e} \text{ (STEPFIX)}$$

Now we can write infinite loops again!

$$\text{fix } x = x \mapsto \text{fix } x = x \mapsto \dots$$

We can also write useful recursive programs.

$$\text{fact} \triangleq \text{fix } \text{fact} = \lambda n : \text{int}. \text{case } n \leq \bar{1} \text{ of } \{x.\bar{1}; y.n * (\text{fact } (n - 1))\}$$

$$\begin{aligned} & (\text{fix } \text{fact} = \lambda n : \text{int}. \text{case } n \leq \bar{1} \text{ of } \{x.\bar{1}; y.n * (\text{fact } (n - 1))\}) \bar{2} \\ \mapsto & (\lambda n : \text{int}. \text{case } n \leq \bar{1} \text{ of } \{x.\bar{1}; y.n * ((\text{fix } \text{fact} = \dots) (n - 1))\}) \bar{2} \\ \mapsto & \text{case } \bar{2} \leq \bar{1} \text{ of } \{x.\bar{1}; y.\bar{2} * ((\text{fix } \text{fact} = \dots) (\bar{2} - \bar{1}))\} \\ \mapsto & \text{case inr } () \text{ of } \{x.\bar{1}; y.\bar{2} * ((\text{fix } \text{fact} = \dots) (\bar{2} - \bar{1}))\} \\ \mapsto & \bar{2} * ((\text{fix } \text{fact} = \dots) (\bar{2} - \bar{1})) \\ \mapsto & \bar{2} * ((\text{fix } \text{fact} = \dots) \bar{1}) \\ \mapsto & \bar{2} * ((\lambda n : \text{int}. \text{case } n \leq \bar{1} \text{ of } \{x.\bar{1}; y.n * ((\text{fix } \text{fact} = \dots) (n - 1))\}) \bar{1}) \\ \mapsto & \bar{2} * (\text{case } \bar{1} \leq \bar{1} \text{ of } \{x.\bar{1}; y.\bar{1} * ((\text{fix } \text{fact} = \dots) (\bar{1} - \bar{1}))\}) \\ \mapsto & \bar{2} * (\text{case inl } () \text{ of } \{x.\bar{1}; y.\bar{1} * ((\text{fix } \text{fact} = \dots) (\bar{1} - \bar{1}))\}) \\ \mapsto & \bar{2} * \bar{1} \\ \mapsto & \bar{2} \end{aligned}$$

2 Recursive Types

If we really want a good model of functional languages, we need more interesting types, in particular to be able to express recursive data structures like linked lists. Let's try to build a type `intlist` which has two introduction forms: `nil` is the empty list, and `cons` $(\bar{n}, l)$ adds $\bar{n}$ to the front of l , where l is another `intlist`. The fact that there are two ways to build it suggests using sum types:

$$\begin{aligned}\text{intlist} &\triangleq \text{unit} + (\text{int} \times \text{intlist}) \\ \text{nil} &\triangleq \text{inl } () \\ \text{cons } (e_1, e_2) &\triangleq \text{inr } (e_1, e_2)\end{aligned}$$

But this definition is circular, so it doesn't work. We need some way of defining recursive types.

$$\tau ::= \dots \mid \alpha \mid \mu\alpha.\tau$$

In $\mu\alpha.\tau$, we define the type we want as τ , in which α is bound to a recursive instance of the type. We call α a type variable: it stands for a type, rather than an expression (and we'll substitute types for it). So, for example:

$$\text{intlist} \triangleq \mu\alpha.\text{unit} + (\text{int} \times \alpha)$$

As an aside, this means we now have a way to make types with unbound type variables, like $\alpha \times \text{unit}$. We need a judgment $\Delta \vdash \tau \text{ ok}$ that says that τ doesn't have unbound type variables. Here, Δ is a context that has the type variables we're allowed to use.

$$\begin{array}{c} \frac{\alpha \in \Delta}{\Delta \vdash \alpha \text{ ok}} \quad \frac{\Delta, \alpha \vdash \tau \text{ ok}}{\Delta \vdash \mu\alpha.\tau \text{ ok}} \quad \frac{}{\Delta \vdash \text{unit ok}} \quad \frac{\Delta \vdash \tau_1 \text{ ok} \quad \Delta \vdash \tau_2 \text{ ok}}{\Delta \vdash \tau_1 \rightarrow \tau_2 \text{ ok}} \quad \frac{\Delta \vdash \tau_1 \text{ ok} \quad \Delta \vdash \tau_2 \text{ ok}}{\Delta \vdash \tau_1 \times \tau_2 \text{ ok}} \\[10pt] \frac{\Delta \vdash \tau_1 \text{ ok} \quad \Delta \vdash \tau_2 \text{ ok}}{\Delta \vdash \tau_1 + \tau_2 \text{ ok}} \end{array}$$

We'll talk more about this a bit later.

So, does that mean we just have, e.g., $\text{nil} \triangleq \text{inl } ()$? Not quite. $\text{inl } ()$ has type $\text{unit} + (\text{int} \times \text{intlist})$, not $\mu\alpha.\text{unit} + (\text{int} \times \alpha)$, which is how we defined `intlist`. So we need a way of converting between these two types.

First, note that we want to perform the following substitution:

$$[\text{intlist}/\alpha](\text{unit} + (\text{int} \times \alpha))$$

or, expanding:

$$[(\mu\alpha.\text{unit} + (\text{int} \times \alpha))/\alpha](\text{unit} + (\text{int} \times \alpha))$$

$$e ::= \dots \mid \text{roll}_\tau e \mid \text{unroll } e$$

$$\frac{\Gamma \vdash e : [\mu\alpha.\tau/\alpha]\tau}{\Gamma \vdash \text{roll}_{\mu\alpha.\tau} e : \mu\alpha.\tau} \text{ } (\mu\text{-I}) \quad \frac{\Gamma \vdash e : \mu\alpha.\tau}{\Gamma \vdash \text{unroll } e : [\mu\alpha.\tau/\alpha]\tau} \text{ } (\mu\text{-E})$$

The `roll` and `unroll` expressions are really just there to make the typing rules work out, but they're in the expressions, so we need step rules for them:

$$\begin{array}{c}
\frac{e \text{ val}}{\text{roll}_\tau e \text{ val}} \text{ (VALROLL)} \qquad \frac{e \mapsto e'}{\text{roll}_\tau e \mapsto \text{roll}_\tau e'} \text{ (STEPSEARCHROLL)} \\
\\
\frac{e \mapsto e'}{\text{unroll } e \mapsto \text{unroll } e'} \text{ (STEPSEARCHUNROLL)} \qquad \frac{e \text{ val}}{\text{unroll roll}_\tau e \mapsto e} \text{ (STEPUNROLL)}
\end{array}$$

Basically, `unroll` just cancels out a `roll`. Here are the correct definitions for int lists:

$$\begin{array}{lcl}
\text{intlist} & \triangleq & \mu\alpha.\text{unit} + (\text{int} \times \alpha) \\
\text{nil} & \triangleq & \text{roll}_{\text{intlist}} (\text{inl } ()) \\
\text{cons } (e_1, e_2) & \triangleq & \text{roll}_{\text{intlist}} (\text{inr } (e_1, e_2))
\end{array}$$

We use `unroll` if we want to use integer lists. For example, the following function returns the first element of a list or 0 if it's empty:

$$\lambda x : \text{intlist}.\text{case unroll } x \text{ of } \{_.\bar{0}; p.\text{fst } p\}$$

We can combine this with the fixed point operator to make a function that adds 1 to every element of a list:

$$\text{fix } f = \lambda x : \text{intlist}.\text{case unroll } x \text{ of } \{_.\text{roll}_{\text{intlist}} \text{ inl } (); p.\text{roll}_{\text{intlist}} \text{ inr } (\text{fst } p + \bar{1}, f (\text{snd } p))\}$$

Note that, to use something of a recursive type, we have to unroll it to turn it into the actual underlying type. Just before we return something of a recursive type, we roll it so that we have the recursive type backs. This pattern works most of the time.

As it turns out, we didn't actually need `fix`: just STLC with recursive types is enough to not only encode general recursion, but actually all of the untyped lambda calculus. This is because we can define the “type” of “untyped” lambda terms:

$$D \triangleq \mu\alpha.\alpha \rightarrow \alpha$$

We have:

$$\begin{array}{ll}
\lambda x : D.(\text{unroll } x) x & : D \rightarrow D \\
\text{roll}_D (\lambda x : D.(\text{unroll } x) x) & : D
\end{array}$$

So we can encode our infinite loop lambda term $(\lambda x.x x)(\lambda x.x x)$ as

$$(\lambda x : D.(\text{unroll } x) x) (\text{roll}_D (\lambda x : D.(\text{unroll } x) x))$$